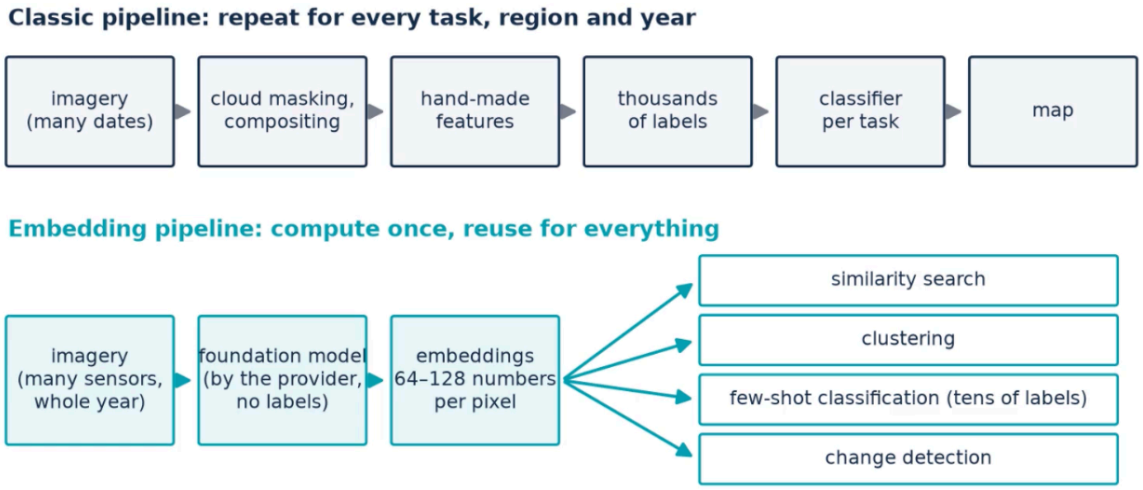
Geo University (EO Embedding Course)

Link: https://www.geo.university/courses/take/earth-observation-embeddings/lessons/78000717-1-1-why-earth-observation-needs-a-new-representation

Chapter 1: Foundations from pixels to vectors

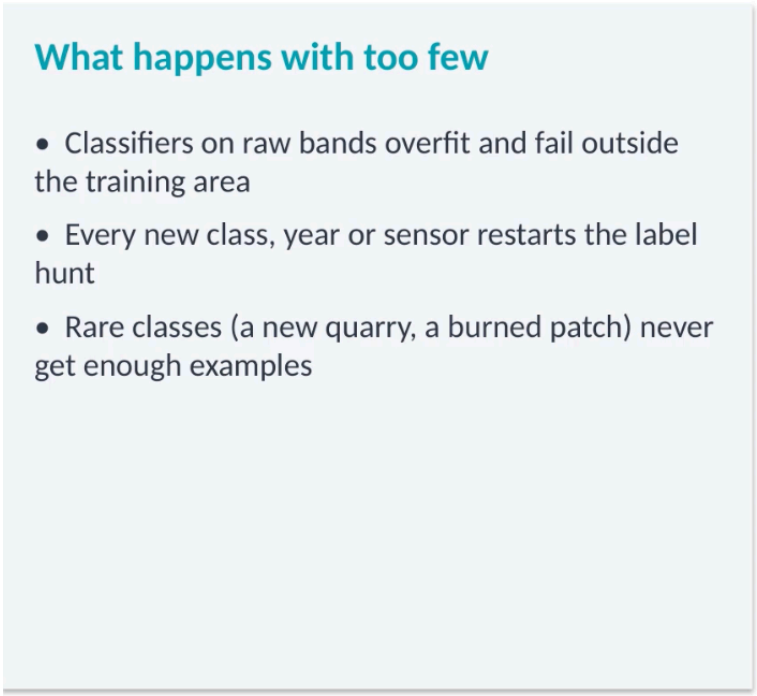
1.1 Why do we need new way to represent satellite data instead of reflectance value?

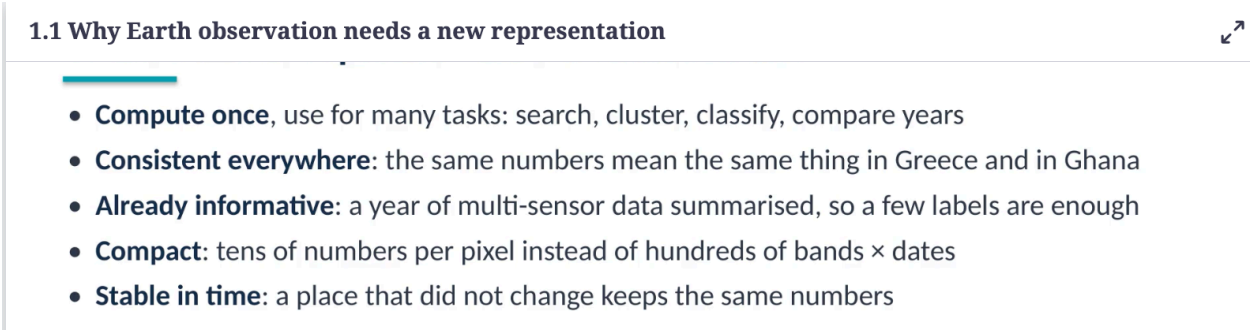
1. Volume of data



2.

3. Too few labels: labels are expensive,





Key takeaways:

- Reflectance tells you how bright a pixel is in each band. An embedding tells you what kind of place it is.
- The bottleneck on the EO is the information extraction not the data acquisition.
- The classic pipeline redo the feature engineering and labelling for every task, region and year
- Labels are scarce source
- Embedding promise: compute once, consistent, compact, informative and stable

1.2 Machine learning essential for EO people

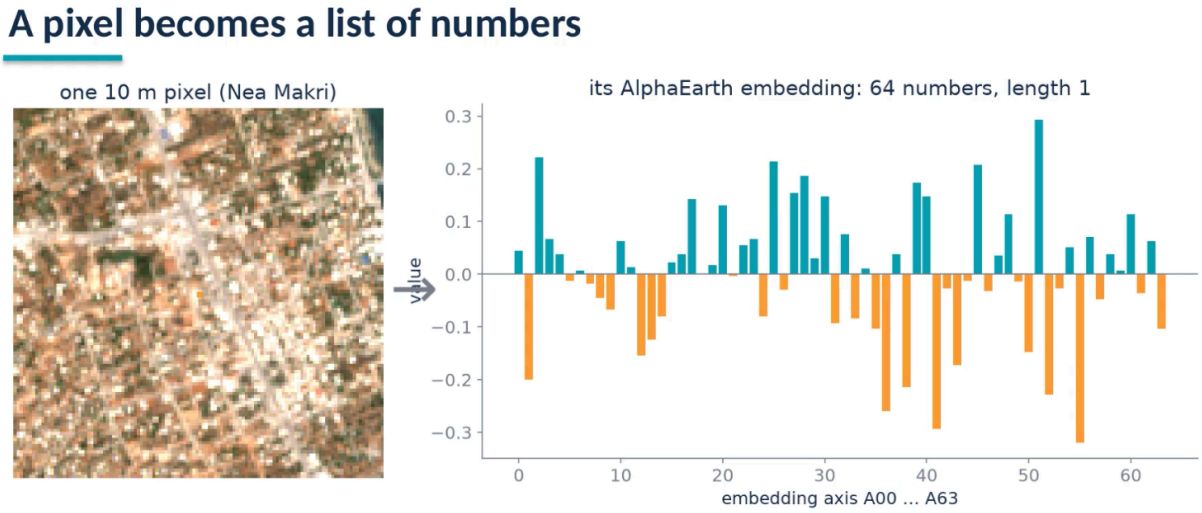
Embedding/ representation/ feature vector: the same thing, seen from three different communities

Linear probe: a simple linear classifier on frozen embeddings, the standard test of how good they are

An embedding model is a network cutoff before its last layer

1.3 What is embeddings?

Embedding is a vector of “d” mdimension



One 10 m pixel in the town of Nea Makri and its AlphaEarth embedding: 64 values between −1 and 1.

A list of “d” dimensions is a vector.

Alpha Earth= 64 dimension

Tessara= 128 dimension

Embedding space: put every pixels vector in the same space

Places seen in similar data gets similar vectors

Two models give two different equally valid spaces: embessding of tessara and alpha earth will be different for same place, so do not mix vectors across model

1.4 EO concepts?

The sensors behind today's embedding products

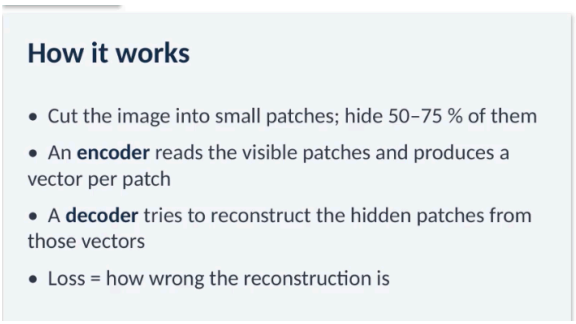
Sensor	Type	Resolution	Revisit	Used by
Sentinel-2 MSI	optical, 13 bands, 443–2190 nm	10–60 m	5 days	AlphaEarth, TESSERA, Major TOM, Clay, Prithvi (via HLS)
Sentinel-1 C-SAR	radar, VV/VH, sees through clouds	10 m (GRD)	6–12 days	AlphaEarth, TESSERA, Major TOM, Clay
Landsat 8/9 OLI	optical, 11 bands	30 m (15 m pan)	16 days	AlphaEarth, Clay, Prithvi (HLS)
Elevation, climate, gravity, text	auxiliary	varies	static	AlphaEarth

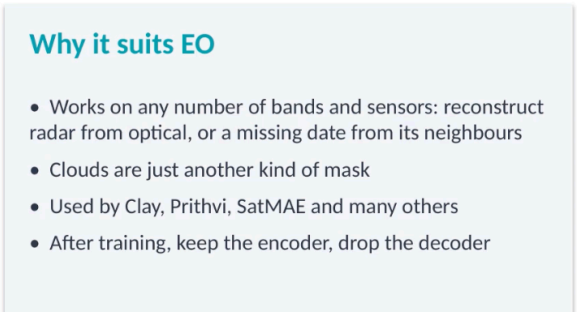
HLS = Harmonized Landsat Sentinel-2, NASA's 30 m merged product.

Chapter 2: How Embeddings are made?

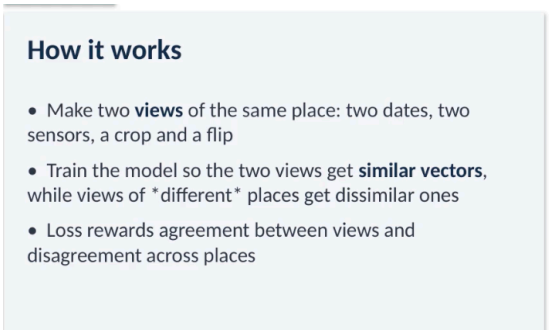
2.1 Self-supervised learning: learning without labels

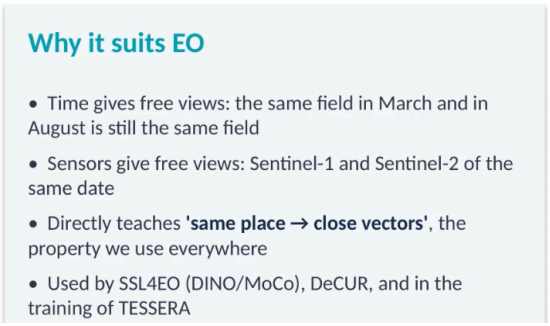
- The puzzle trick: take an image, hide or distort part of it, ask the model to recover what was hidden
- To solve the puzzle model must learn what the world looks like: textures, shapes, seasons.
- The puzzle is thrown away after the training: what we keep is the representation the model built to solve it: this representation is called embedding
    - Puzzle 1: masked autoencoder (MAE)





  - Puzzle 2: Contrastive learning (two views of the same place should get close vectors)





- What the model learns?
  - learns what co-occurs in the data: spectra, texture, seasonality, context
  - Learns the statistics of its training data: regions, seasons, or sensors. That were rare in the training are represented less as well
  - Learns to ignore what did not help the puzzle whicj may include what you care about (rare, small, fast)

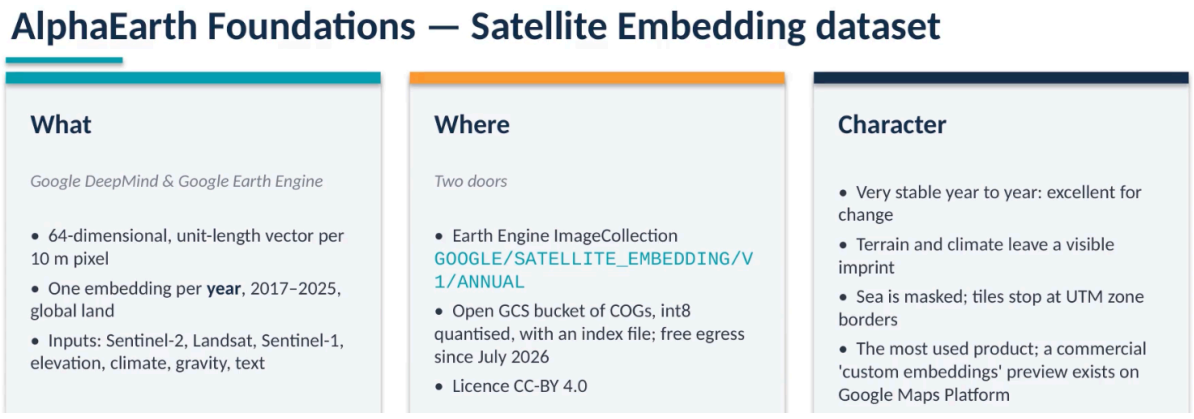
2.2 Anatomy of an EO Foundation Models

- Step 1 : From pixels to tokens
  - The image is cut into small square patches, each patch is flattened into one vector: called token
  - A 256 pixel chip with 8-pixel patches gives 32*32 =1024 tokens
    • **Chip dimensions:** The chip is 256 * 256 pixels.
    • **Patch size:** Each patch is 8 * 8 pixels.
    The Calculation:

1. **Width**= 32 patches across
2. **Height=** 32 patches down
3. **Total Tokens**= 1,024 tokens

- Step 2: transformer encoder
  - Transformer is a stack of layers in which every token looks at every other token (attention) and update itself
  - After many layers each tokens vector encodes not only its own patch but its context.
- Step 3: Telling the model where, when and what it sees
  - Model must told each token position in the image (positional encoding)
  - EO models add more: time (day of year , lat/long)
  - Clay adds wavelength per band, so it can handle sentinel2, radar
  - Alpha-earth feeds elevation, climate and text descriptions alongside imagery
  - These encodings are why an embedding knows about season, terrain, and geography
- Step 4: What comes out
  - pixel-level (alpha=earth, tessara), patch level (clay, Major TOM), patch tokens (grid of vectors per chip)

2.3 Product landscape



TESSERA

What

University of Cambridge and partners

- 128-dimensional vector per 10 m pixel; v2 is Matryoshka (16 to 128)
- Annual, 2017–2025 for Europe and the US, 2024 global
- Inputs: Sentinel-1 and Sentinel-2 time series only

Where

geotessera

- `pip install geotessera`; 0.1° tiles, NumPy or GeoTIFF or Zarr
- Model weights and training pipeline are open; reproducible on standard cloud
- Licence **CC0**: no attribution required, commercial use fine

Character

- Sharper land-cover separation in the first components; sea is embedded, not masked
- Vectors not unit length: normalise before cosine
- Runs on AMD GPUs; the most 'academic-open' of the products

Major TOM embeddings

What

ESA Φ-lab with CloudFerro

- One vector per 384-pixel **patch** (~3.8 km) of Sentinel-2 or Sentinel-1
- Four embedding sets: SSL4EO (S2 multispectral), SSL4EO (S1 radar), DINOv2 (S2 RGB), SigLIP (S2 RGB)
- 170 million+ embeddings from 8 million images, 2016–2023, per acquisition

Where

Hugging Face

- GeoParquet, ~1 GB files sorted by latitude, under `Major-TOM/`
- Computed on CloudFerro's CREODIAS GPUs, mirrored there
- Licence CC-BY-SA 4.0

Character

- Landscape-scale, dated: 'what kind of area is this, on this day'
- SigLIP set enables **text search** (Hands-on 6)
- Comes with a FAISS index and a viewer app
- The **Major TOM grid** is becoming a standard for exchanging embeddings

Clay — an open model you run yourself

What

Clay Foundation, formerly under Radiant Earth

- Vision transformer trained with masked autoencoding on Sentinel-2, Sentinel-1, Landsat, NAIP, LINZ, MODIS
- Any sensor via band wavelengths; time and location inputs
- Outputs a class token per chip and a token per 8-pixel patch

Where

GitHub & Hugging Face

- Code and weights under **Apache-2.0**
- v1.5 checkpoint (5 GB) on Hugging Face; older v0 embeddings on Source Cooperative
- Runs on a laptop CPU at ~0.3 s per chip

Character

- Not a dataset: **you choose the date, sensor and area**
- The reference open model for teaching and fine-tuning
- Hands-on 7

Also on the map

Name	Who	What it is	Access
Earth Index	Earth Genome	Global patch embeddings from the SoftCon model (384-d), 2024; built for a click-to-search app aimed at journalists and NGOs (e.g. finding mines)	GeoParquet on Source Cooperative, CC-BY
Prithvi-EO-2.0	NASA & IBM	Multi-temporal foundation model on 30 m Harmonized Landsat Sentinel-2; 300 M and 600 M variants; strong on floods, burn scars, crops	Weights on Hugging Face; TerraTorch
TerraMind	IBM & ESA	Multimodal any-to-any model (optical, radar, DEM, land cover, text); can generate a missing modality	Weights on Hugging Face; TerraTorch
Embed2Scale	EU Horizon Europe consortium	Research project on compressing EO and weather data cubes into embeddings; public CVPR challenge	embed2scale.eu
SSL4EO, DeCUR, MMEarth, DOFA...	Academic	Pre-trained encoders used to compute several of the above; the raw material of the field	GitHub, Hugging Face

Comparison

	AlphaEarth	TESSERA	Major TOM	Clay v1.5
Unit	10 m pixel	10 m pixel	3.8 km patch	2.6 km chip + 80 m tokens
Dimensions	64, unit length	128 (v2: 16–128)	1024–1152, raw	1024, raw
Time	annual 2017–2025	annual 2017–2025 (EU/US), 2024 global	per acquisition 2016–2023	any date you choose
Inputs	S1, S2, Landsat, DEM, climate, text	S1, S2	S2 RGB / S2 MSI / S1	any: S2, S1, Landsat, aerial...
Format	int8 COG on GCS; Earth Engine	npy / GeoTIFF / Zarr via geotessera	GeoParquet on Hugging Face	you compute it
Licence	CC-BY 4.0	CC0	CC-BY-SA 4.0	Apache-2.0
Best for	change, few-shot maps	land cover, open reuse	landscape search, text search	custom dates and sensors

2.4 Precomputed embeddings vs running the model yourself

Two ways to get a vector

Precomputed products

- AlphaEarth, TESSERA, Major TOM, Earth Index
- Download, read, use. Minutes to first result
- No GPU, no model expertise, no imagery handling
- Provider fixes: sensors, dates, resolution, model version

Run a model yourself

- Clay, Prithvi, TerraMind, SSL4EO encoders
- Fetch imagery, preprocess, run inference. Hours to first result
- CPU for small areas; GPU or cloud for regions and time series
- You fix: sensors, dates, resolution, and you can fine-tune

When precomputed is enough (most of the time)

- Your question is **annual or slower**: land cover, land-use change, urban growth, forest loss, recovery
- Your inputs are Sentinel and Landsat; you do not need a special sensor
- You need **consistency** across a country or a decade more than control over dates
- You have no GPU budget and want results this week
- Rule of thumb: if AlphaEarth or TESSERA can answer it, start there; you can always graduate later

When you need to run the model

- **Timing**: you need a specific date, or monthly cadence, and per-date products do not cover your area or years
- **Sensor**: PlanetScope, aerial, drone, hyperspectral, or a radar mode the products do not use
- **Resolution**: sub-metre imagery, or a patch size the products do not offer
- **Fine-tuning**: your task is hard and you have enough labels to adapt the model
- **Freshness**: the annual product for this year does not exist yet
- **Sovereignty or auditability**: you must control the model version and where the data is processed

The hybrid options

Option	What it is	Trade-off
Google Custom Satellite Embeddings	AlphaEarth model run on demand on your area and dates (Maps Platform preview, near real time)	commercial, Google-hosted; AlphaEarth quality without waiting for the annual release
CloudFerro GeoAI on CREODIAS	Embedding generation and anomaly detection on Copernicus data, next to the archive	European infrastructure, no data egress; service maturing
Compute-to-data (Earth Engine, CDSE, Planetary Computer)	Run Clay or Prithvi where the imagery already is	you still manage the model, but skip the download
Fine-tune on top of precomputed	Train a small model on downloaded embeddings plus your labels	cheapest form of adaptation; cannot change what the embedding saw

Key takeaways:

- Adapting a small model on top of pre-computed embeddings is the cheapest customization
- Matryoshka embeddings: Vectors trained so that their first k dimensions already form a usable embedding

Chapter 3: Where and How to access Embeddings?

3.1 Access landsacpe

Where the bytes live

Object storage

GCS, S3, Azure Blob

- AlphaEarth on Google Cloud Storage; Earth Index and Clay v0 on Source Cooperative (S3)
- Public buckets: HTTPS, no account, **range requests** fetch only the bytes you need
- Files: Cloud Optimized GeoTIFF, GeoParquet

Hugging Face

the ML community's data hub

- Major TOM embeddings, Clay and Prithvi weights, SigLIP
- Git-like repos of large files; `huggingface_hub` downloads and caches
- DuckDB reads Parquet there directly with `hf://` paths

Platforms & libraries

compute where the data is

- Google Earth Engine (AlphaEarth as ImageCollection)
- `geotessera` (TESSERA), TorchGeo and TerraTorch loaders
- CDSE / CREODIAS for Copernicus-side processing

Formats you will meet

Format	What it holds	Used by	Read with
Cloud Optimized GeoTIFF (COG)	raster tiles with internal tiling and overviews; band per embedding axis	AlphaEarth (64 bands, int8)	rasterio / GDAL, windowed reads over HTTPS
GeoParquet	a table with a geometry column; one row per patch or point	Major TOM, Earth Index	DuckDB, GeoPandas, pyarrow
NumPy .npy + scales	raw arrays per tile, int8 with per-pixel scale factors	TESSERA	geotessera (dequantises for you)
Zarr	chunked N-dimensional arrays, cloud native	TESSERA (alternative), your own stores	xarray, zarr
Earth Engine ImageCollection	server-side rasters, no download	AlphaEarth	Earth Engine API

Two habits that make big products small

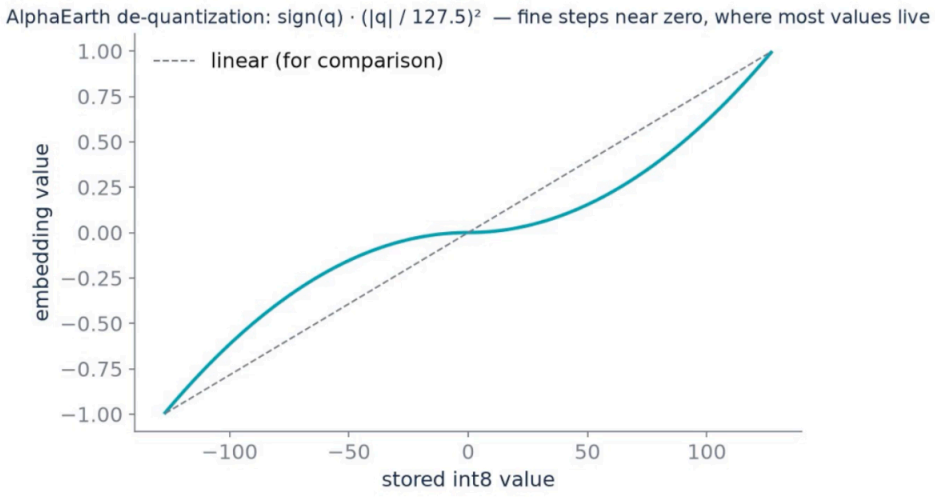
- **Read windows, not files.** A COG lets you fetch a 10 × 10 km window (60 MB) out of a 300 MB tile; the rest never leaves the bucket
- **Read overviews when you can.** Every COG carries downsampled copies; at 80 m a whole region is a few tens of megabytes (Hands-on 3)
- **Use the index.** AlphaEarth ships a table of every file's footprint; Major TOM's files are sorted by latitude; TESSERA has a registry. Never list a bucket by hand
- **Cache.** Download once, keep it next to the notebook

Quantization

quantization, which is a compression technique used by AlphaEarth to reduce file sizes by shrinking high-precision numbers down to compact **8-bit signed bytes** (integers ranging from roughly -128 to 127)

it's a clever trick to compress massive geospatial embedding files into 8-bit formats without noticeably losing accuracy where it matters most.

Quantisation: why the files are 8-bit



AlphaEarth stores each of the 64 values as a signed byte and maps it back with a squared curve, spending precision where values are dense.

3.2 AlphaEarth: earth engine anf open GCS bucket

Two doors to the same data

Earth Engine

- `ee.ImageCollection('GOOGLE/SATELLITE_EMBEDDING/V1/ANNUAL')`
- One image per year and UTM tile, 64 bands A00–A63, already float
- Server-side: sample, classify, cluster without downloading
- Needs an Earth Engine account; free for non-commercial use

Open GCS bucket

- `gs://alphaearth_foundations/satellite_embedding/v1/annual/`
- Cloud Optimized GeoTIFFs, int8, same 64 bands
- Plain HTTPS, **no account**, free egress since July 2026
- Read with rasterio, GDAL, QGIS, anything

How the bucket is organised

```
gs://alphaearth_foundations/satellite_embedding/v1/annual/
  aef_index.parquet        # footprint of every file: year, zone, path, bounds (300 k rows,
70 MB)
  aef_index.csv / .gpkg     # same index, other formats
  README.md                 # de-quantisation formula, conventions
  2017/ ... 2025/           # one folder per year
    34N/                    # one folder per UTM zone (120 in total)
      xmc2wlvtjehg2if5c-0000008192-0000008192.tiff   # 8192 x 8192 px, 64 bands, int8, ~300
MB
      ^ Earth Engine image id     ^ row offset ^ col offset within that image
```

- Each Earth Engine image (16 384 px) is split into four COGs so no file exceeds ~300 MB
- Tiles stop at UTM zone boundaries; the same place in two years has **different file names**, so always go through the index

Find the tiles with DuckDB

```
import duckdb
con = duckdb.connect(); con.execute("INSTALL httpfs; LOAD httpfs;")
url = "https://storage.googleapis.com/alphaearth_foundations/satellite_embedding/v1/annual/
aef_index.parquet"
tiles = con.execute(f"""
    SELECT path, utm_zone, crs, wgs84_west, wgs84_south, wgs84_east, wgs84_north
    FROM read_parquet('{url}')
    WHERE year = 2024
      AND wgs84_east  >= 23.90 AND wgs84_west  <= 24.00     -- bbox (west, south, east, north)
      AND wgs84_north >= 38.00 AND wgs84_south <= 38.10
""").df()
```

- DuckDB reads only the row groups it needs from the remote Parquet file
- Tiles overlap across zone borders: keep the zone that contains the centre of your area

Read a window with rasterio

```
import rasterio
from rasterio.warp import transform_bounds
from rasterio.windows import Window

opts = {"GDAL_NUM_THREADS": "ALL_CPUS", "GDAL_HTTP_MULTIRANGE": "YES",
        "GDAL_HTTP_MERGE_CONSECUTIVE_RANGES": "YES", "GDAL_DISABLE_READDIR_ON_OPEN": "EMPTY_DIR"}
url = tiles.path[0].replace("gs://", "https://storage.googleapis.com/")

with rasterio.Env(**opts), rasterio.open(url) as ds:
    x0, y0, x1, y1 = transform_bounds("EPSG:4326", ds.crs, 23.90, 38.00, 24.00, 38.10)
    r0, c0 = ds.index(x0, y0); r1, c1 = ds.index(x1, y1)          # bbox corners -> pixel rows/cols
    win = Window(min(c0, c1), min(r0, r1), abs(c1 - c0), abs(r1 - r0))
    raw = ds.read(window=win)                                     # int8, shape (64, rows, cols), ~8 s
```

- The GDAL options are the difference between 8 seconds and 90 seconds for the same 60 MB window
- Rows in these files run **south to north**; flip the array (and the transform) if you need north-up

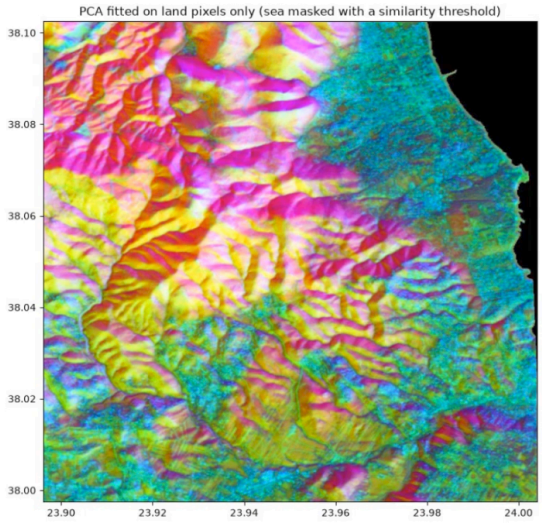
De-quantise, and check

```
import numpy as np

def dequantize(raw):
    v = raw.astype(np.float32)
    out = np.sign(v) * (np.abs(v) / 127.5) ** 2      # documented mapping: int8 -> [-1, 1]
    out[raw == -128] = np.nan                        # -128 is the nodata value, in every band at once
    return out

emb = dequantize(raw)
norms = np.linalg.norm(emb, axis=0)                  # should be 1.0 everywhere there is data
print(np.nanmin(norms), np.nanmedian(norms), np.nanmax(norms))   # 0.991  1.0  1.009
```

- Unit-length vectors: cosine similarity between two pixels is just their dot product
- NaN propagates: keep it, and mask before you fit anything

What you will do in Hands-on 1

- Check your environment, define the study area (East Attica), find its tile in the index
- Read the 2024 window straight from the bucket and de-quantise it
- Look at the numbers, the no-data mask, and a first PCA false-colour picture
- Compare four places with cosine similarity, put the picture on a web map, save the data for later



What you will produce: AlphaEarth 2024 over East Attica, three principal components as RGB.

Earth Observation Embeddings · Chapter 3 · Where and how to access embeddings · AlphaEarth: Earth Engine and the open GCS bucket 7

Key takeaways:

- DuckDB find the tiles
- Set GDAL threads and range merging: 8s instead of 90s

3.3 TESSERA, Major TOM, Earth Index in python

TESSERA with geotessera

```
from geotessera import GeoTessera
gt = GeoTessera(embeddings_dir="data/cache/tessera")        # where downloaded tiles are kept

bbox = (23.90, 38.00, 24.00, 38.10)                          # (west, south, east, north)
tiles = gt.registry.load_blocks_for_region(bounds=bbox, year=2024)
# -> [(2024, 23.95, 38.05), ...]  0.1-degree tiles named by their centre; neighbours included

for year, lon, lat, arr, crs, transform in gt.fetch_embeddings(tiles):
    print(arr.shape, arr.dtype, crs)                         # (1138, 913, 128) float32 EPSG:32634

mosaic, transform, crs = gt.fetch_mosaic_for_region(bbox, year=2024, target_crs="EPSG:32634")
values = gt.sample_embeddings_at_points([(23.95, 38.05)], year=2024)   # for a few points
```

- The library keeps a registry of 4.4 million tiles, downloads what you ask for and de-quantises it
- Array layout is `(rows, cols, 128)`: the embedding axis comes **last**, unlike rasterio's `(bands, rows, cols)`

TESSERA details that bite

- Tiles are stored in the **UTM zone of the tile**: a region crossing 24° E returns arrays in two coordinate systems
- Vectors are **not unit length** (norms around 30): normalise before cosine similarity
- The sea is embedded, not masked; no-data appears as zero vectors
- Coverage: 2017–2025 for Europe and the US, 2024 elsewhere; check the coverage map, request missing tiles on GitHub
- Version 2 (Matryoshka, float32) is published; the library still serves v1 tiles by default in 2026

TESSERA details that bite

Major TOM on Hugging Face

```
# 46 Parquet files of ~1 GB, sorted by latitude. Which ones cover 36.5-41.1 N? Ask the footers only:
import duckdb
con = duckdb.connect(); con.execute("INSTALL httpfs; LOAD httpfs;")
url = "hf://datasets/Major-TOM/Core-S2RGB-SigLIP/embeddings/part_03001-03100.parquet"
con.execute(f"SELECT min(stats_min), max(stats_max) FROM parquet_metadata('{url}') "
            f"WHERE path_in_schema = 'centre_lat'").fetchone()        # ('36.478893', '38.7908')

# Download the files you need once (cached), then filter locally:
from huggingface_hub import hf_hub_download
path = hf_hub_download("Major-TOM/Core-S2RGB-SigLIP", "embeddings/part_03001-03100.parquet", repo_type="dataset")
df = con.execute(f"""SELECT grid_cell, timestamp, centre_lat, centre_lon, geometry, embedding
                     FROM read_parquet('{path}')
                     WHERE centre_lat BETWEEN 37.6 AND 38.6 AND centre_lon BETWEEN 23.0 AND 24.6""").df()
```

- Columns: `unique_id`, `grid_cell`, `timestamp`, `product_id`, `centre_lat/lon`, `geometry` (WKB), `utm_footprint`, `embedding` (list of 1152 floats)
- One row per 384-pixel patch per Sentinel-2 acquisition; embeddings are raw, normalise them

Earth Index on Source Cooperative

```
# GeoParquet per UTM grid cell and date range, e.g. Barcelona / 31TDF, 2024:
url = "https://data.source.coop/earthgenome/earthindexembeddings/<path-to>/31TDF_2024-01-01_2025-01-
01.parquet"
gdf = gpd.read_parquet(url)                # point geometries + 384-float embeddings (SoftCon model)
gdf.head()
```

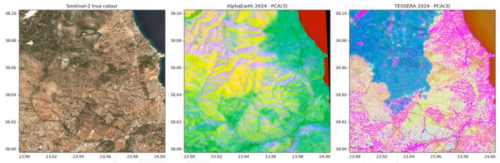
- Source Cooperative is an S3-backed repository for open geospatial data; files are plain HTTPS
- Same recipe as Major TOM: find the cell, read the Parquet, normalise, search
- Built for the Earth Index application (Earth Genome): click a place, find similar places worldwide
- CC-BY 4.0; global for 2024

Same task, three products: what changes in the code

	AlphaEarth	TESSERA	Major TOM / Earth Index
Find data	DuckDB on index Parquet	geotessera registry	Parquet footers / file naming
Read	rasterio window over HTTPS	geotessera fetch (npy tiles)	DuckDB / GeoPandas on Parquet
In memory	int8 → float32 (64, H, W)	float32 (H, W, 128)	table, one row per patch
Normalise?	already unit	yes	yes
No-data	−128 → NaN, sea masked	zero vectors, sea present	missing rows
Grid	UTM zone of the tile	UTM zone of the tile	polygons in WGS 84 + UTM footprint

What you will do in Hands-on 2

- Fetch the TESSERA tile for East Attica and put it on the AlphaEarth grid
- Compare the two products: PCA pictures, similarity maps from the same points, and how well their distances agree
- See the practical differences with your own eyes: norms, sea handling, tile borders



Same area, same year: Sentinel-2, AlphaEarth and TESSERA.

Key takeaways

- TESSERA: a Python library, 0.1° tiles in local UTM, embedding axis last, normalise before use
- Major TOM: Parquet on Hugging Face sorted by latitude; read footers to pick files, download once, filter with DuckDB
- Earth Index: GeoParquet per UTM cell on Source Cooperative, same recipe
- Only the access code differs between products; the analysis code is the same

3.4 European side: CDSE, CREODIAS and CloudFerro GeoAI

Copernicus Data Space Ecosystem (CDSE)

- The official access point for Copernicus data since 2023, run for ESA by a consortium led by **T-Systems**, with **CloudFerro** operating the cloud infrastructure, and Sinergise, VITO and others
- Full Sentinel archive online, free: STAC and OData catalogues, S3 access, openEO, Sentinel Hub APIs, Jupyter
- Public cloud processing next to the archive: no download, no egress
- The natural place to run Clay, Prithvi or TerraMind on Sentinel data at European scale

CREODIAS and the Major TOM embeddings

- **CREODIAS** is CloudFerro's commercial cloud with the Copernicus archive mounted locally (EODATA)
- In December 2024 CloudFerro and ESA Φ-lab released the first global open embedding dataset for Copernicus imagery: **Major TOM embeddings**
- 170 million+ embeddings from 62 TB of Sentinel-1 and Sentinel-2 data, computed on CREODIAS GPUs (NVIDIA L40S)
- Four models: SSL4EO (S2 multispectral, S1 radar), DINOv2 and SigLIP (S2 RGB); MMEarth and DeCUR followed
- Published on Hugging Face and mirrored in the CREODIAS repository; CC-BY-SA

CloudFerro GeoAI: embeddings as a service

- Announced services on CREODIAS: **embedding generation** for satellite imagery with a choice of models, **anomaly detection** on time series of embeddings, curated AI-ready datasets
- Runs inside the CREODIAS environment: your data and the Copernicus archive never leave European infrastructure
- Aimed at agencies and companies that need fresh or custom embeddings without building a GPU pipeline
- Status (Sept 2026): documentation is thin; treat it as a service to evaluate, not yet a commodity

Key takeaways

- CDSE is the official, free access point for Copernicus data, with processing next to the archive
- The Major TOM embeddings were computed by CloudFerro and ESA Φ-lab on CREODIAS: European infrastructure, open licence
- CloudFerro GeoAI offers embedding generation and anomaly detection as services; evaluate before relying on them
- Compute-to-data and sovereignty make embeddings the exchange format of choice for large archives

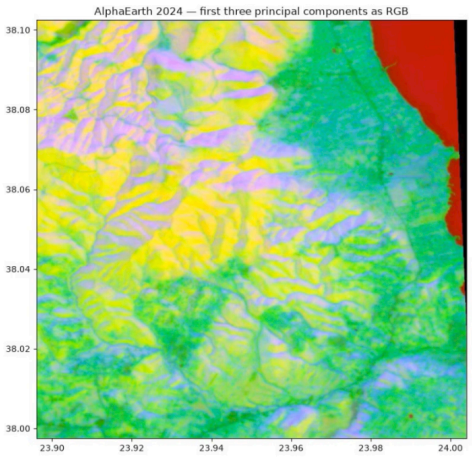
Chapter 4: Working with Embeddings?

4.1 Seeing embedding space

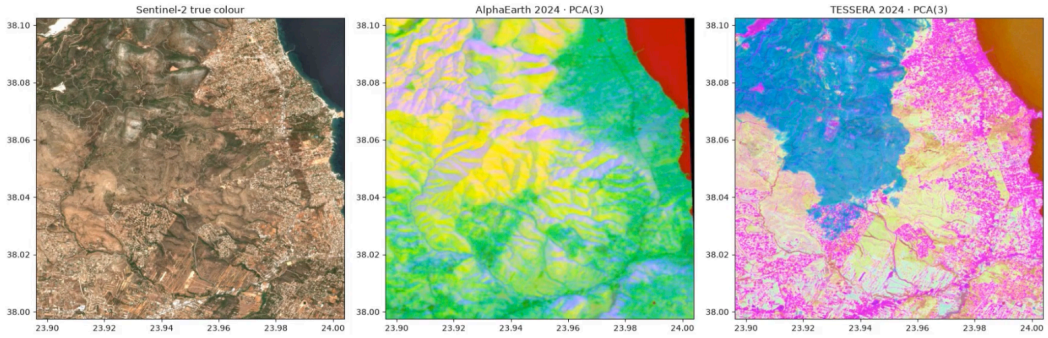
- raster with 64/128 bands can not be displayed
- Any picture of an embedding is a projection. 64 numbers squeezed into 3 with information lost

Principal component analysis (PCA)

- Finds the directions in the 64-D space along which the pixels **vary most**; the first three become red, green and blue
- Linear, fast (a second for a million pixels), deterministic given the data
- Fit it on a **sample** (50 000 pixels is plenty), then apply to all
- Stretch each component between its 2nd and 98th percentile before mapping to 0–255
- To compare two images (two years, two products), **fit one PCA on both** and apply it to each



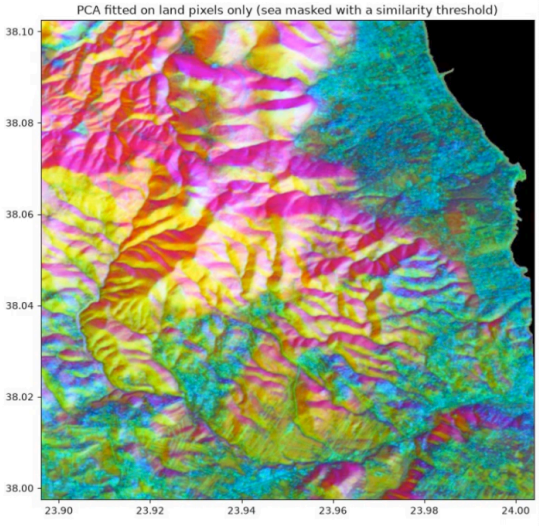
The largest variation is not always what you care about



Same window, same year. AlphaEarth's first components follow relief and coastline; TESSERA's follow land cover.

Tricks that make PCA pictures more useful

- **Mask the dominant class** before fitting: exclude sea (by nodata or by similarity to a sea pixel) and land gets the whole colour budget
- **Fit on a bigger area** than you display, so local quirks do not define the axes
- **Show components 4–6** as a second picture: often where subtle land-cover differences live
- **Do not compare colours across windows or years** unless the PCA was shared



UMAP and friends: non-linear projections

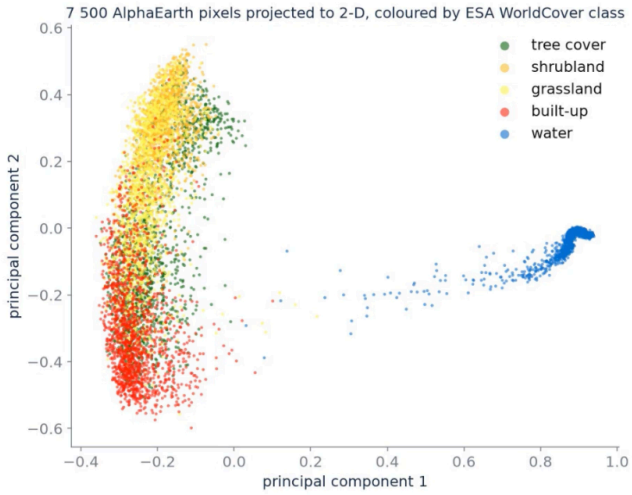
What they do

- UMAP and t-SNE place points in 2-D so that **neighbours stay neighbours**, bending the space as needed
- Clusters become visually separate blobs, much clearer than in PCA
- Great for scatter plots of a sample (10 000–100 000 points), with colours from labels or clusters

What to watch

- Distances *between* blobs mean little; only local neighbourhoods are preserved
- Stochastic: another run, another picture
- Slow for millions of pixels; do not use for full rasters
- `umap-learn` in Python; expect 30 s for 50 000 points on a laptop

Scatter plots: the honest way to check separability



7 500 pixels, PCA to 2-D, colours from an independent land-cover map. Overlap here predicts confusion in a classifier.

Key takeaways

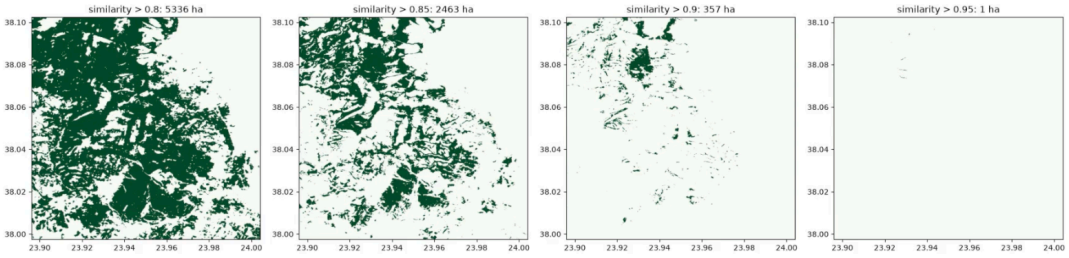
- Every embedding picture is a projection; the projection depends on the data in the window
- PCA: fast, linear, fit on a sample, stretch by percentiles, share it when comparing
- The first components show the largest variation, which may be terrain or sea, not your classes
- UMAP for scatter plots of samples; not for rasters, not for distances between blobs
- Colour a scatter plot with independent labels to judge separability before you train anything

4.2 Similarity search: find more like us

The mechanics

- Query vector **q**, unit length. For every pixel vector **x**: similarity = **q** · **x** (cosine, because both have length 1)
- One matrix–vector product: a million pixels × 64 dimensions in well under a second
- Euclidean distance carries the same information for unit vectors: $d^2 = 2 - 2 \cdot \cos$
- Output: a similarity map in [−1, 1]; in practice most land pixels score 0.3–0.9 against any land query

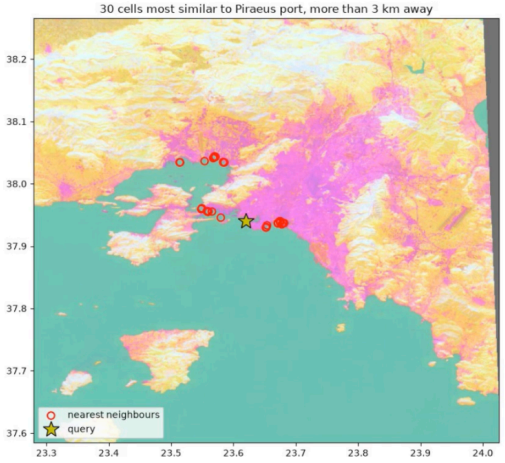
From map to layer: the threshold is your decision



The same similarity map at four thresholds. Hectares change by a factor of four.

Scaling up: vector indexes

- One window is a matrix product. A country is hundreds of millions of vectors and many queries: you want an **index**
- **FAISS** (Meta): exact `IndexFlatIP` for a few million vectors, ~1 ms per 100 000 vectors per query
- Approximate indexes (IVF, HNSW) trade a little recall for orders of magnitude in speed and memory; Major TOM ships one for 170 M patches
- Databases: PostgreSQL + `pgvector`, or dedicated vector stores, when search must sit next to your other data
- Exclude the query's own neighbourhood, or the answer is just itself



Attica at 80 m, 720 000 vectors: the 30 cells most similar to Piraeus port are the

Choosing the resolution of the search

Resolution	How	Vectors for Attica (3 800 km²)	Good for
10 m	full-resolution window	38 million	objects: greenhouses, quarries, small plots
80 m (factor 8)	COG overview, mean embedding per 8×8 block	600 000	regional: ports, airports, lakes, urban fabric
3.8 km patch	Major TOM patch embeddings	~400	landscape types; text search (Chapter 5)

Key takeaways

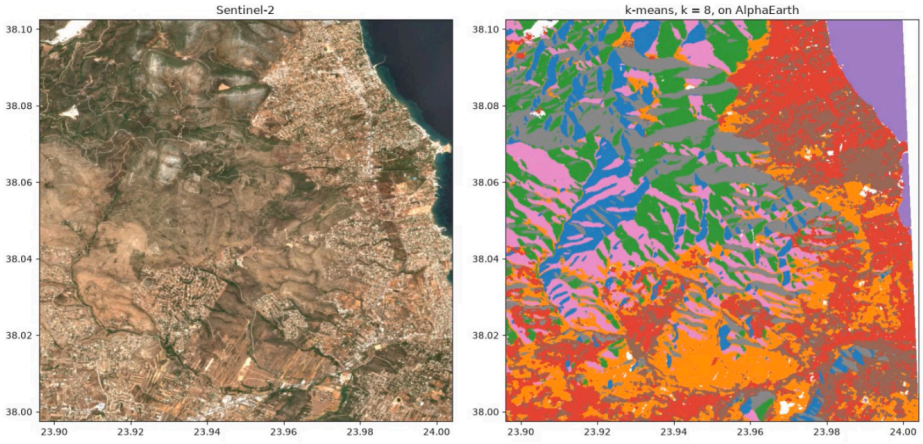
- Similarity = dot product of unit vectors; a window is one matrix product
- Query with a prototype of several examples, never a single pixel
- Threshold, clean, polygonise: the threshold is a documented decision, not a constant
- Negative examples sharpen a query without losing recall
- Vector indexes (FAISS, pgvector) scale search to regions; match resolution to object size

4.3 Clustering: unsupervised land cover

k-means in one slide

- Pick *k* centres; assign each pixel to the nearest centre; move each centre to the mean of its pixels; repeat
- Fit on a **sample** (50 000 pixels), then assign all pixels: seconds, not minutes
- Distance is Euclidean; for unit vectors that is equivalent to cosine, so no normalisation issue with AlphaEarth
- Result: an integer per pixel, 0 … k−1, and *k* centre vectors, which are themselves prototypes you can reuse for search

What it looks like



East Attica, k = 8 on AlphaEarth 2024. Colours are arbitrary cluster ids.

Naming the clusters

- Overlay on imagery and click through: the fastest, and often enough
- **Cross-tabulate** with an existing map (WorldCover, CORINE, cadastre): which class dominates each cluster?
- Pure clusters (water, built-up, dense forest) take the map's name; mixed clusters are the interesting ones
- A cluster that splits one map class in two may be **real detail the map lacks** (irrigated vs rain-fed, dense vs open shrub)
- Merge clusters that you cannot tell apart; the map is for humans

Choosing k

Rules of thumb

- Start with **2× the number of classes you expect**, then merge
- Too few: distinct things share a cluster (fields with grassland)
- Too many: one thing is split by illumination or slope
- The 'elbow' of inertia vs k is rarely sharp for embeddings; do not agonise over it

Alternatives

- **Gaussian mixtures**: soft membership, useful for mixed pixels
- **HDBSCAN**: finds the number of clusters itself and marks noise; slower, good on samples
- **Hierarchical**: cluster the k-means centres again to get a two-level legend
- Cluster **PCA-reduced** vectors (10–20 components) for speed with no visible loss

Cluster map or classification?

Cluster map when...

- You have no labels and need a first stratification of an unfamiliar region
- You want to **discover** what kinds of places exist, not impose a legend
- You need strata for sampling: pick training points per cluster to cover the diversity
- The product is for analysts, who will name and merge

Classification when...

- The legend is fixed by the client or the regulation
- You have even a few labels per class (Lesson 4.4)
- The product must be comparable to other maps or years
- Tip: cluster first, then label one pixel per cluster — the cheapest training set you will ever build

4.4 Few-shot classification: labels are cheap now

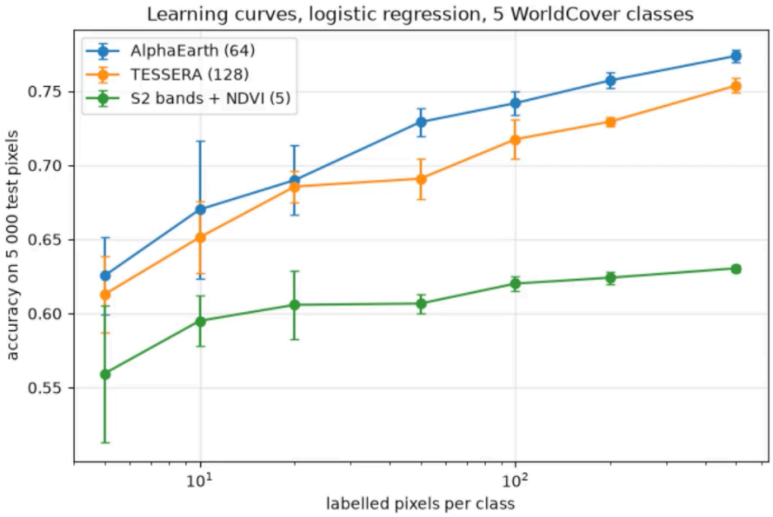
Why a linear model is enough

- The foundation model already did the non-linear work: in embedding space, classes are close to **linearly separable**
- So the classifier can be simple: **logistic regression**, or k-nearest neighbours, or a small random forest
- Simple models need few examples, train in a second, and do not overfit 64 dimensions with 100 points the way a deep network would
- This setup is called a **linear probe**: the standard way to measure how good an embedding is

Where do labels come from?

- **Your own clicks** on imagery: 10 per class in an afternoon; often the most trustworthy
- **Existing maps**: ESA WorldCover, CORINE, national land-use, OpenStreetMap; plentiful, but dated and ~75 % accurate themselves
- **Field data**: gold standard, expensive; now you need far less of it
- **Cluster then label**: one label per k-means cluster covers the diversity of the area (Lesson 4.3)
- Whatever the source, labels are noisy: part of your 'error' is the reference being wrong

How many labels do you need?



East Attica, five WorldCover classes, logistic regression. Error bars over five random draws of training pixels.

Validate like a geographer, not like a data scientist

- Neighbouring pixels are almost identical: a random train/test split **leaks** and inflates accuracy by 10–20 points
- Use a **spatial split**: checkerboard blocks (1 km), separate regions, or leave-one-area-out
- Balance the test set per class, or accuracy just measures the majority class
- Report **per-class** metrics (F1, confusion matrix), not one number
- If your labels come from a map, say so: you are measuring agreement with that map, not truth

Practical recipe

- Standardise features; logistic regression with default regularisation; that is the baseline
- Start with 20 labels per class; add labels where the confusion matrix hurts
- Try a random forest only if the linear probe is clearly failing; if it helps a lot, question your labels
- Predict the whole raster in one call; a million pixels take a second
- Keep the model: it transfers to next year's embeddings of the same product (check on a few points first)

Key takeaways

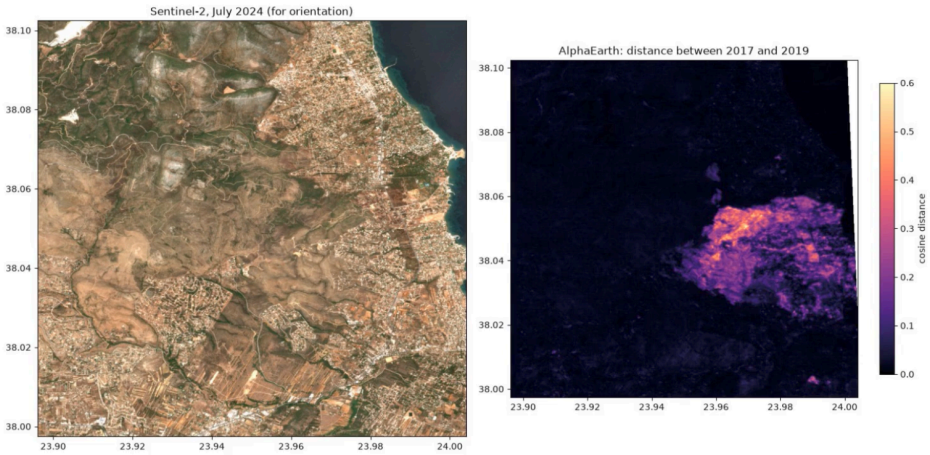
- Embeddings make classes nearly linearly separable, so a linear probe on few labels works
- Both embedding products beat raw bands at every label budget; curves flatten after a few hundred labels
- Validate spatially, per class, and remember the reference map is itself noisy
- The confusion matrix tells you what to label next
- One trained model, reused every year: a map series for the cost of one afternoon of labelling

4.5 Change detection across years

The method

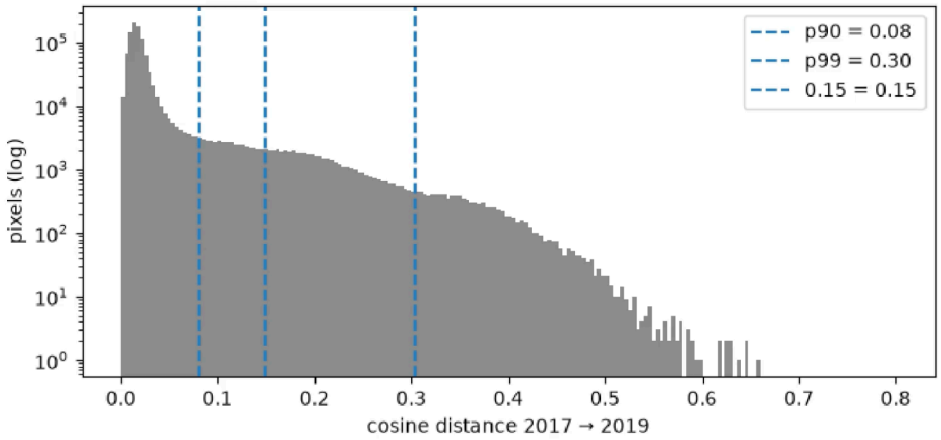
- For each pixel: distance = 1 − cos($\mathbf{e}_{t1}$, $\mathbf{e}_{t2}$). Zero means same kind of place; values near 1 mean nothing in common
- Works because embeddings of **unchanged places are stable** across years: median year-to-year distance ≈ 0.02 in our area
- One line of NumPy for a million pixels; no co-registration, no radiometric normalisation, no cloud masking
- The output is a change *magnitude*, not a change *type*

Mati, 2017 → 2019



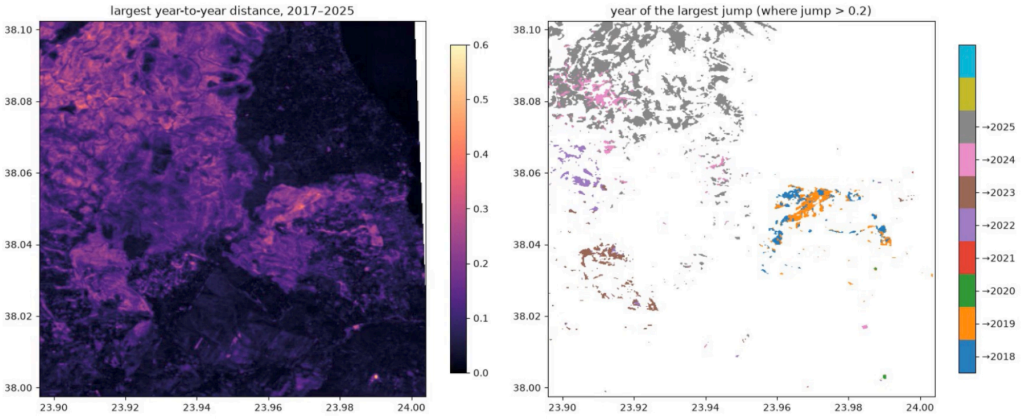
Cosine distance between the 2017 and 2019 AlphaEarth embeddings. The fire of 23 July 2018 stands out with no tuning.

Thresholds: find the noise floor first



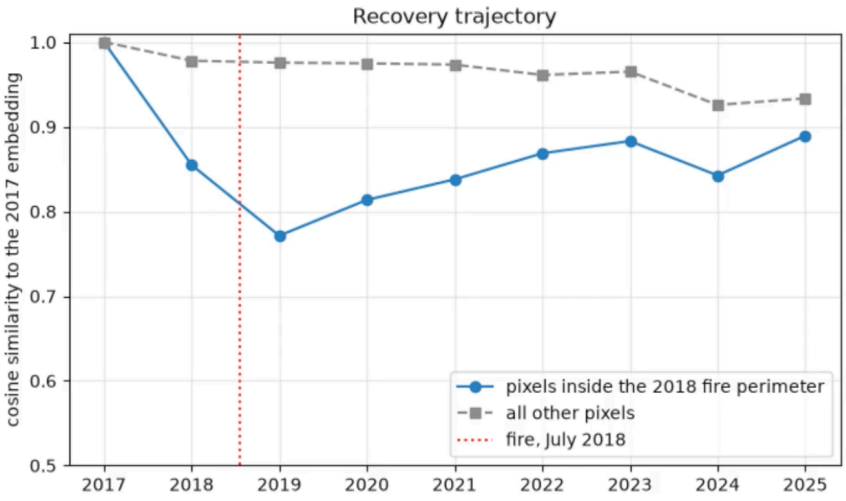
Distribution of distances. Half the pixels moved less than 0.02, 90 % less than 0.08; the tail is real change.

Nine years in one figure: when did each pixel change most?



Left: largest year-to-year distance 2017–2025 at 20 m. Right: the year of that jump, for pixels that jumped by more than 0.2.

Recovery: following the burned pixels



Mean similarity to the 2017 embedding, for pixels inside the fire perimeter and for all other pixels.

What can go wrong

- **Slow change** (sprawl, degradation) never crosses a yearly threshold: compare distant years or fit a trend
- **Drift without change**: a drought year or a new satellite moves *all* embeddings a little; judge each pixel against its year's distribution
- **Events inside the year** are diluted; monthly products or per-date embeddings if timing matters
- **A distance is not a label**: classify before and after (Lesson 4.4) or look at the imagery to say what changed
- **Product versions**: never compare embeddings from two model versions; re-run the series when the product updates

Key takeaways

- Change = cosine distance between annual embeddings; stable products make it work
- Compare the year before with the year after an event; the event year is diluted
- Find the noise floor in the histogram, then threshold, clean, polygonise
- Consecutive-year distances give a 'when did it change' inventory; trajectories show recovery
- Magnitude is not type; slow change and drift need different treatment

5. Patch level embeddings and text search with Major TOM

5.1 Patch level embeddings and open models

Pixel vs patch

A patch embedding summarises a whole chip: landscape, not location.

Major TOM embeddings, as a table

- One row per patch per Sentinel-2 acquisition: `grid_cell`, `timestamp`, `product_id`, footprint, `embedding`
- Nine patches per ~10 km grid cell, 10 % overlap; dates 2016–2023, **one date at a time**, not annual
- Four embedding sets from four models: SSL4EO on multispectral S2 and on S1 radar, DINOv2 and **SigLIP** on S2 RGB
- SigLIP embeddings: 1 152 raw floats; normalise before cosine
- The course ships the Attica rows (1 376 patches, 3.5 MB); the full set is 46 GB on Hugging Face
- **Plot before you trust**: over Attica the SigLIP embeddings vary by grid row as much as by landscape, an artefact of the production pipeline (Hands-on 6, exercise 4)

Vision-language models: images and sentences in one space

- SigLIP (Google) was trained on hundreds of millions of **image–caption pairs**: an image tower and a text tower learn to give matching pairs similar vectors
- So a sentence can be embedded into the *same space* as an image, and compared with cosine similarity
- Major TOM used SigLIP's image tower on Sentinel-2 RGB patches; you use the text tower on your query
- **Zero-shot**: no EO training data was involved at any point; the model knows 'lake' from the web
- Absolute similarities are low (0.15–0.25); only rankings matter

What works and what does not

Top-10 Major TOM patches around Attica for six sentences. Landscape-scale queries work; object-scale queries do not.

Zero-shot classification and other uses

- A set of sentences becomes a legend: assign each patch the class whose text vector is closest → a landscape map from eight phrases
- **Image-to-image**: the patch containing Piraeus retrieves the other industrial waterfronts; robust, no text needed
- **Time**: several dates per cell in the full dataset allow per-date landscape change at 4 km scale
- **Pre-selection**: rank a country's patches by a query, then send the top few hundred to a pixel-level method or a human
- EO-specific vision-language models (RemoteCLIP, SkyCLIP…) narrow the domain gap; Major TOM's other sets have no text tower

Key takeaways

- Patch embeddings describe landscapes, not locations: 3.8 km, one date at a time
- SigLIP puts images and sentences in one space, so Sentinel-2 patches can be searched with text, zero-shot
- Landscape-scale queries work; object-scale queries fail for reasons of granularity and domain
- Text legends give zero-shot landscape maps; image queries and pre-selection are the robust everyday uses

5.2 Running Clay v1.5 on you r laptop

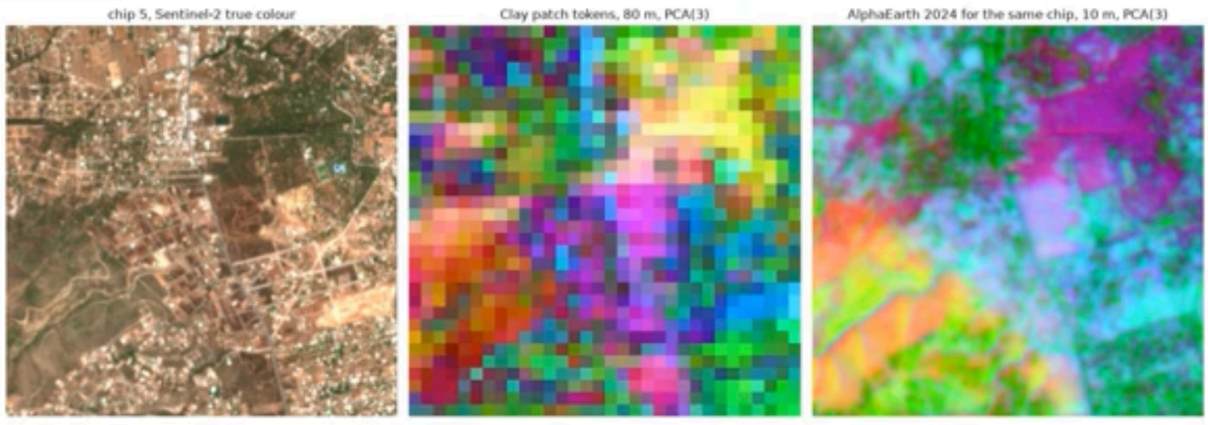
What you need

- `pip install git+https://github.com/Clay-foundation/model.git` (brings PyTorch Lightning)
- The checkpoint `clay-v1.5.ckpt` from Hugging Face: **5.2 GB, once**; ~8 GB free RAM to load it
- `metadata.yaml`: per sensor, the band order, wavelengths, means and standard deviations for normalisation
- Imagery: Sentinel-2 L2A chips of **256 × 256 pixels** at 10 m from a STAC catalogue (Element 84 Earth Search on AWS, no account)
- A CPU. Inference is ~0.3 s per chip once the model is loaded

The input dictionary

```
batch = {
    "pixels": torch.tensor((chips - MEANS) / STDS),   # (N, 10 bands, 256, 256), normalised reflectance
    "time":   torch.tensor([[sin(week), cos(week), sin(hour), cos(hour)] ...]),   # (N, 4)
    "latlon": torch.tensor([[sin(lat), cos(lat), sin(lon), cos(lon)] ...]),       # (N, 4)
    "waves":  torch.tensor([0.493, 0.560, 0.665, 0.704, 0.740, 0.783, 0.842, 0.865, 1.610, 2.190]),  # µm per band
    "gsd":    torch.tensor(10.0),                      # ground sampling distance in metres
}
with torch.no_grad():
    tokens, *_ = module.model.encoder(batch)         # (N, 1 + 32*32, 1024)
cls   = tokens[:, 0]                                  # one 1024-d vector per chip
patch = tokens[:, 1:].reshape(-1, 32, 32, 1024)       # one 1024-d vector per 80 m patch
```

Two outputs: a vector per chip, and a grid of vectors inside it



One chip: Sentinel-2 true colour, Clay's 32 × 32 patch tokens as PCA colours, and AlphaEarth at 10 m for the same area.

What it costs to scale

	12 chips	Attica (3 800 km²)	Greece (132 000 km²)
chips of 2.56 km	12	~600	~20 000
CPU time at 0.3 s per chip	4 s	3 min	2 h
Sentinel-2 to download, 10 bands, one date	15 MB	0.8 GB	26 GB
...for 12 dates a year		10 GB, 36 min	300 GB, 24 h

A GPU does 50–100 chips per second; processing next to the archive removes the download entirely.

When running the model yourself earns its cost

- A specific **date** or a monthly cadence that no per-date product covers for your area
- A **sensor** the products do not use: PlanetScope, aerial, drone, hyperspectral
- **Fine-tuning** on your labels for a hard task (Clay and Prithvi are designed for it; TerraTorch helps)
- **Freshness**: this year's annual product is not out yet
- Otherwise: start with a precomputed product and graduate only where needed

Key takeaways

- Clay v1.5 runs on a laptop CPU: 5 GB checkpoint once, ~0.3 s per 256-px chip after loading
- Inputs: normalised pixels, time and location as sine/cosine, wavelengths per band, GSD
- Outputs: a class token per chip and a 32 × 32 grid of 80 m patch tokens
- One date over a country is feasible on a laptop; dense time series belong on GPUs next to the archive
- Run it yourself for dates, sensors and fine-tuning; otherwise use precomputed products

Chapter 6: Applications, limits, and what comes next

6.1 Applications in practice

Agriculture

Crop type mapping with few labels

- Annual embeddings encode phenology; linear probes reach accuracies close to full random-forest pipelines with 5–10 % of the labels
- Field boundaries and parcel homogeneity visible in similarity maps

Monitoring compliance

- Common Agricultural Policy checks: flag parcels whose embedding does not match the declared crop's prototype
- Year-to-year distance flags land-use change on declared parcels

Caveats

- Annual granularity hides double cropping and harvest dates
- 10 m is coarse for small Mediterranean parcels; patch products are far too coarse

Forests and carbon

Forest type and structure

- Embeddings separate forest types and densities that single-date indices merge
- Canopy height and biomass regressed from embeddings with sparse lidar or plot data (TESSERA and AlphaEarth papers report strong gains over spectral baselines)

Disturbance and recovery

- Fire scars, clear-cuts and windthrow as year-to-year distance; recovery as trajectories (Hands-on 5)
- Consistent across a whole country in one run

Caveats

- Slow degradation needs trend analysis, not thresholds
- Regressions are only as good as the reference measurements' geolocation

Urban and socio-economic

Urban form

- Local climate zone mapping: AlphaEarth and TESSERA embeddings tested across Swiss cities with few labels (2026 study)
- Building height inferred from embeddings alone, first results 2026

Socio-economic proxies

- Embeddings as plug-and-play features for poverty, population and economic activity estimates (AEF-Econ, 2026)
- Replaces bespoke feature engineering on night lights and imagery

Caveats

- Correlations learned in one country may not transfer to another; validate locally
- Ethical questions around inferring wealth from space are not solved by better features

Disasters, water and biodiversity

Disasters

- Fire and flood extent from before/after distance when annual products suffice
- Rapid mapping needs per-date embeddings: run Clay or Prithvi on the post-event scene (Prithvi-EO-2.0 is benchmarked on floods and burn scars)

Water and wetlands

- Seasonal water and wetland types separate well in annual embeddings thanks to radar input
- Reservoir and lake change across years

Biodiversity and habitats

- Habitat mapping with few labels; embedding diversity as a proxy for landscape heterogeneity
- Cambridge's TESSERA work started from conservation use cases

Search and monitoring at scale

- **Earth Index** (Earth Genome): journalists and NGOs click on a mine, a feedlot or an airstrip and find all similar sites in a country; used in investigations of illegal mining in the Amazon
- **Anomaly detection** on embedding time series: CloudFerro's GeoAI and others flag places whose vector moved unexpectedly
- **Inventory building**: rank a country by similarity to a prototype, verify the top hits, iterate with negatives (Lesson 4.2)
- **Consistency checks** on existing maps: pixels whose embedding disagrees with their map class are where the map is likely wrong

Patterns across the applications

- Wins are largest where **labels are scarce** and the signal is **annual or slower**
- Embeddings replace feature engineering, not domain knowledge: the class definitions, the validation design and the interpretation are still yours
- Most operational uses pair embeddings with a **simple model** (linear probe, kNN, thresholds), not with more deep learning
- The literature is 2025–2026 preprints and first operational pilots; expect revisions

Key takeaways

- Agriculture, forests, urban, disasters, water, biodiversity: embeddings are being used across the board, mostly with linear probes and few labels
- Search and anomaly detection at country scale are the genuinely new capabilities
- Annual granularity and 10 m resolution set the boundaries; per-date and finer needs mean running a model
- Domain knowledge, validation and interpretation remain the analyst's job

6.2 Limitations and pitfalls

Limits of the products

- **Temporal granularity**: annual means annual. Harvest dates, floods that recede, temporary works are invisible or diluted
- **Spatial resolution**: 10 m is the floor; individual trees, narrow roads and small buildings are mixed pixels, as always
- **What the model saw**: rare surfaces (salt pans, glaciers, new materials) and rare regions are represented less well
- **Quantisation**: int8 storage costs ~1 % on vector length; negligible for search and classification, worth remembering for subtle regressions
- **Masked classes**: AlphaEarth masks the sea; a water study needs TESSERA or a different product

Domain shift

- A classifier trained on Attica embeddings applied in Thessaly meets different soils, crops and climate: accuracy drops even though the embedding is global
- The **embedding** transfers; the **decision boundary** you drew may not. Retrain with a few local labels
- Across years the drift is small but real: check a handful of stable points before applying last year's model
- Across **products** nothing transfers: AlphaEarth and TESSERA are different spaces

Interpretability, or the lack of it

- No axis means anything nameable; you cannot say 'the map is red because NDVI is low'
- Explanations come from **examples and comparisons**: show the prototype pixels, show the nearest neighbours, show the imagery
- Regulators and clients may require a traceable rule; embeddings support a traceable *procedure* (data, model version, labels, threshold), not a rule
- Keep the classic indices alongside for explanation and sanity checks

The analyst's pitfalls

- **Over-reading PCA colours** as classes, or comparing colours across windows and years
- **Random train/test splits** on autocorrelated pixels: accuracies that evaporate in the next valley
- **Single-pixel queries** and untuned thresholds transferred from another product or area
- **Treating a distance as a diagnosis**: every change is not a fire; every anomaly is not fraud
- **Trusting the reference**: WorldCover, CORINE and cadastres are 70–85 % accurate; your 'error' partly measures theirs

Licences, versions, reproducibility

Product	Licence	What it means in practice
AlphaEarth	CC-BY 4.0	attribute Google and Google DeepMind in any derived map or paper
TESSERA	CC0	no conditions; commercial use and redistribution free
Major TOM embeddings	CC-BY-SA 4.0	attribute, and share derived data under the same licence; check before using in closed products
Earth Index	CC-BY 4.0	attribute Earth Genome
Clay model & weights	Apache-2.0	free use, modification and redistribution; your embeddings are yours

Record the product version and file checksums with every analysis; a new model version is a new embedding space.

Dependence, cost and energy

- Free today: Google's egress waiver (July 2026), Hugging Face hosting, Cambridge's grants. **None is guaranteed forever**; keep local copies of what you depend on
- Open weights (Clay, TESSERA) mean you could recompute; a closed model (AlphaEarth) means you could not
- Training a foundation model costs megawatt-hours; using its embeddings costs almost nothing. The economics only work if the embeddings are widely reused
- For public bodies: prefer products with open weights or European hosting where sovereignty matters (Lesson 3.4)

Key takeaways

- Products: annual granularity, 10 m floor, training-data statistics, masked classes
- Domain shift: embeddings transfer, decision boundaries may not; retrain locally, never mix products
- Interpretability comes from examples, not axes; keep indices for explanation
- Analyst pitfalls: PCA over-reading, random splits, single-pixel queries, distance as diagnosis, trusting the reference
- Read the licence, record the version, keep local copies

6.3 Building an embedding-powered workflow

The shape of the system

- **Ingest**: pull the product for your extent once per release (AlphaEarth windows, TESSERA tiles, Major TOM rows) → local store
- **Store**: a cloud-native array store for rasters, a Parquet table for patches; record version and checksums
- **Index**: a vector index for search; a plain classifier for maps
- **Labels**: a small, versioned table of points with class, source, date and author
- **Products**: maps, change layers, search results, each stamped with product version, labels version, thresholds
- **Repeat** next year: new embeddings in, same models and thresholds, diff against last year

Storage choices

Data	Format	Why	Size for Greece at 10 m
pixel embeddings, one year	Zarr (chunked, int8 or float16) or the original COGs	windowed access, cloud-native, lazy loading with xarray	~85 GB int8 (AlphaEarth), ~170 GB (TESSERA int8)
pixel embeddings, 9 years	one Zarr with a time axis	trajectories and change in one array	~0.75 TB int8
patch embeddings	GeoParquet	table semantics, joins with cadastre or parcels, DuckDB	~1 GB (Major TOM, Greece)
labels	GeoPackage / GeoParquet, versioned in git or DVC	small, human-edited, must be auditable	kilobytes
models	pickle / ONNX + a JSON card (product version, labels version, metrics)	reproducibility	kilobytes

Indexing and serving

Search

- FAISS flat index up to a few million vectors; IVF or HNSW beyond (Greece at 80 m: 20 M vectors, fits in RAM)
- PostgreSQL + **pgvector** when search must join with attributes and serve a web app
- Store the index next to the vectors; rebuild on every product release

Maps

- Logistic regression or kNN on standardised embeddings; predict per chunk, write COG
- Change: cosine distance per chunk between two years' arrays
- Run as a scheduled job (cron, Airflow, or a simple Makefile): compute is minutes to hours per country on CPU

Labels and humans in the loop

- Start from clusters or similarity search to **propose** candidates; humans confirm, do not draw
- Every label carries: class, source (field, image, map), date, author, confidence
- Re-validate a fixed panel of points every year: it measures product drift *and* landscape change
- Disagreements between the model and the reference map go to a review queue; that queue is your most valuable dataset

Failure modes to design for

- The **product changes**: new model version, new tiling, bucket policy. Pin versions, mirror what you depend on, alert on checksum changes
- The **provider stops**: keep the last good release; prefer products with open weights
- **Silent drift**: a yearly panel of stable points catches it before users do
- **Scale surprises**: memory blows at 10× area. Chunk everything from day one; never load a country into RAM
- **Licence conflicts**: derived data from CC-BY-SA products inherits the licence; decide early

Key takeaways

- Ingest once per release, store cloud-native, index for search, version labels and models, repeat yearly
- Zarr or COGs for pixels, GeoParquet for patches, pgvector or FAISS for search
- Humans confirm candidates; a fixed validation panel measures drift
- Pin, mirror, chunk, and read the licence

6.4 The road ahead

Five directions

- **Higher cadence**: monthly or per-acquisition pixel embeddings; Google's custom embeddings preview and per-date products point the way
- **Multimodal**: models that jointly embed imagery, radar, elevation, weather and text (TerraMind, AlphaEarth's text input); text queries on pixel products
- **Standards**: the Major TOM grid, TorchGeo and TerraTorch loaders, STAC extensions for embeddings; one API for many products
- **Compression as a product**: Embed2Scale's idea of shipping embeddings instead of data cubes; archives that serve vectors natively
- **Open and sovereign**: CC0 weights, European hosting, reproducible training pipelines (TESSERA's recipe on standard clouds)

What to do next

- **Do the capstone**: your own area, a few-shot map and a change map, half a page of interpretation
- **Replace one step** in an existing project with embeddings (the features, or the change layer) and compare
- **Follow the sources**: the AlphaEarth, TESSERA and Major TOM pages, the Clay docs, the TorchGeo release notes
- **Go deeper** if you liked hands-on 7: fine-tuning with TerraTorch, Prithvi and TerraMind on your labels
- **Share**: labels, prototypes and thresholds for your region are worth publishing; the embeddings are already shared